Programming Worlds

PROGRAMMING WORLDS ACADEMY

Git & GitHub 101

From Zero to Your First Pull Request

BEYOND CODE. BEYOND LIMITS.

04

MODULE 04

Git Add, Commit, and History

Choose what to save, save it with a clear message, and read the history you created.

MODULE 04 · LESSON 4.1

Understanding git add

فهم git add

Learning objectives

- Explain what staging means.
- Stage everything or only specific files.
- Tell `git add .` from `git add -A`.
- Inspect what is staged.
- Explain why secrets must never be committed.

Prerequisites

Module 3 and the `my-first-project` folder.

Introduction

Git does not save every change automatically. First you choose which changes belong in the next snapshot. That choice is called staging, and `git add` is the command for it.

Explanation

Staging means placing changes in the staging area, the waiting room for the next commit. `git add` never creates a commit. It only prepares one.

`git add index.html` stages one file. `git add index.html style.css` stages two. `git add .` stages every change in the current folder and its subfolders: new, modified and deleted files.

`git add -A` stages all changes in the whole repository, wherever you are inside it. From the project root, `.` and `-A` do the same thing. They differ when you run them inside a subfolder.

`git add -p` lets you stage piece by piece. Git shows each changed block (a *hunk*) and asks `Stage this hunk [y,n,q,a,d,s,e,?]?`. Answer `y` for yes, `n` for no, and `q` to quit.

Why stage selectively? If you fixed a bug and also started a new feature, two commits are clearer than one. Specific files or `-p` let you split the work.

Inspect. `git status` lists staged files under “Changes to be committed”. `git diff --staged` shows exactly what is staged. If you edit a file **after** staging it, that file

appears in both lists: the commit takes the version you staged, not your newer edit. Run `git add` again to include it.

Secrets. Never stage files with passwords, API keys or `.env` data. Once committed they stay in history. Check `git status` before `git add .`.

In VS Code. Open Source Control (`Ctrl+Shift+G`). The + next to a file stages it. The + next to **Changes** stages all of them (like `git add .`). The − unstages.

Real-world analogy

> Staging is choosing items for a photo. You arrange the objects you want in the frame first. The shutter (the commit) comes after.

Diagram

Illustration · Git Bash (not a real screenshot)

```
$ git add index.html
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
        style.css
```

Figure 1 · Illustration of the expected output

Commands

`git add index.html`
`git add index.html style.css`
`git add .`
`git add -A`
`git add -p`
`git status`

Options: `.` current folder; `-A` all changes in the repo; `-p` choose hunks interactively. Figure 1 shows `git status` after staging one file.

Common mistakes and solutions

Thinking git add saves the work.	Nothing is saved until you commit.
Using git add . without looking.	Run `git status` first so no secret or junk file slips in.
Editing after staging and committing without adding again.	Run `git add` again, or the new edit stays out of the commit.

Practical exercise

> In `my-first-project`, stage only `index.html`. Run `git status`. Then run `git diff --staged`.

Expected result

`index.html` is under “Changes to be committed” and the other two files are still untracked, as in Figure 1.

Knowledge check

1. Does git add create a commit?

- ○ Only the first time
- ○ No, it only stages
- ○ Yes

2. Which command shows what is staged?

- ○ git diff –staged
- ○ git push
- ○ git init

3. Why not commit a .env file?

- ○ Git cannot read it
- ○ It may contain secrets that stay in history
- ○ It is too large

Answers

- No, it only stages — Staging prepares changes. Commit creates the snapshot.
- git diff –staged — It compares the staging area with the last commit.
- It may contain secrets that stay in history — Anything committed remains in history.

Summary

git add stages changes for the next commit. Stage specific files for clean commits, check with git status and git diff –staged, and never stage secrets.

Key vocabulary

Staging area	Waiting room for the next commit
git add -A	Stage all changes in the repository
Hunk	A changed block of a file

Next lesson

Lesson 4.2 — Understanding git commit. Your changes are staged. Now you save them.

MODULE 04 · LESSON 4.2

Understanding git commit

فهم git commit

Learning objectives

- Explain what a commit is.
- Create a commit with a good message.
- Read a commit hash.
- Tell good from bad commit messages.

Prerequisites

Lesson 4.1.

Introduction

A commit is the moment you save a snapshot into history. This is the heart of Git.

Explanation

A **commit** is a snapshot of the staged changes plus a message, an author and a date. It is stored in your local history and linked to the commit before it.

`git commit -m "Add homepage"` creates a commit with that message. `-m` means “message”. Keep the message inside quotes.

Git prints a line such as `[main (root-commit) 3f2a9c1] Add homepage`. **main** is the branch, **root-commit** marks the very first commit, and **3f2a9c1** is the start of the **commit hash**, a unique ID.

Commits stay on your computer. Nothing is on GitHub until you push (Module 5).

Good messages start with a verb and say what changed: `Add homepage`, `Fix navigation styling`, `Update README`. **Bad messages** say nothing: `changes`, `stuff`, `asdf`.

Meaningful commits. Each commit should be one logical change that works. That makes history readable and mistakes easy to undo.

Real-world analogy

> A commit is a save point in a video game with a label. Good labels let you find “before the boss fight” quickly.

Diagram

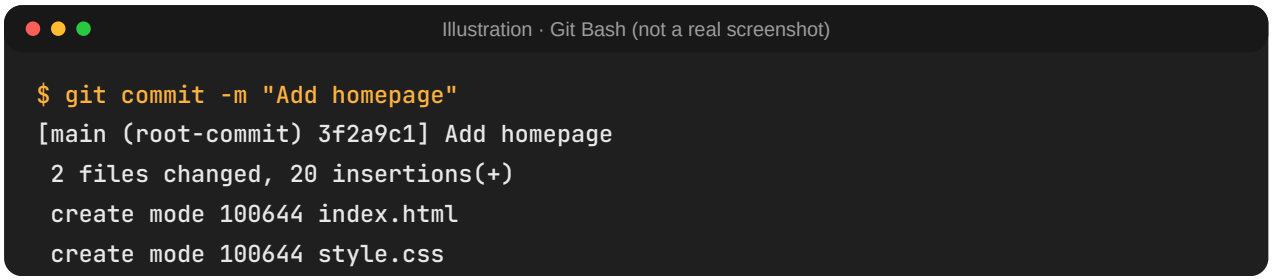


Figure 1 · Illustration of the expected output

Commands

```
git commit -m "Add homepage"
git commit -m "Fix navigation styling"
git commit -m "Update README"
```

Option: `-m` message. Run only after staging. Figure 1 shows the first commit of the project.

Common mistakes and solutions

“Author identity unknown”	Set `user.name` and `user.email` (Lesson 2.3).
“nothing to commit”	Nothing is staged. Run `git add` first.
Forgetting the quotes.	Put the message in double quotes.

Practical exercise

> Commit the staged `index.html` and `style.css` with the message `Add homepage`.

Expected result

Git prints a line starting `[main (root-commit)` and lists two created files, as in Figure 1.

Knowledge check

1. What does -m mean in git commit -m?

- ○ Move
- ○ Message
- ○ Merge

2. Which is a good message?

- ○ Add homepage
- ○ stuff
- ○ asdf

3. Where does a new commit live?

- ○ Nowhere
- ○ On your computer
- ○ On GitHub

Answers

- Message — It supplies the commit message.
- Add homepage — It starts with a verb and says what changed.
- On your computer — Push is the step that uploads it.

Summary

A commit saves staged changes with a message. Use clear messages, and remember commits are local until you push.

Key vocabulary

Commit hash	A unique ID for a commit
root-commit	The first commit in a repository
-m	Option to give the commit message

Next lesson

Lesson 4.3 — Reviewing Git history. You can now save snapshots. Next you will read them.

MODULE 04 · LESSON 4.3

Reviewing Git history

مراجعة سجل Git

Learning objectives

- Read the commit list with `git log`.
- Compare changes with `git diff` and `git diff --staged`.
- Inspect a commit with `git show`.

Prerequisites

Lesson 4.2.

Introduction

History is only useful if you can read it. These commands show what happened and when.

Explanation

`git log` lists commits, newest first. Each entry shows the full hash, author, date and message. If the list is long, Git opens a viewer: press **q** to leave it.

`git log --oneline` shows one short line per commit: a short hash and the message. `(HEAD → main)` marks where you are now.

`git diff` shows changes you made but have **not staged**. `git diff --staged` shows changes that are staged for the next commit. Lines starting with `+` were added and lines starting with `-` were removed.

`git show` displays the latest commit with its changes. `git show 3f2a9c1` shows a chosen commit.

`git status` remains your first check: it tells you what to do next.

Real-world analogy

> `git log` is the logbook of a ship. `git diff` compares today's page with what is already written.

Diagram

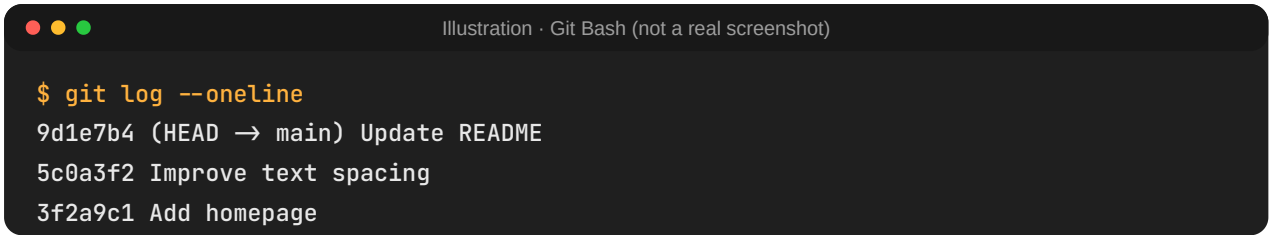


Figure 1 · Illustration of the expected output

Commands

`git log`
`git log --oneline`
`git diff`
`git diff --staged`
`git show`
`git status`

Figure 1 shows `git log --oneline` after three commits.

Common mistakes and solutions

Stuck inside git log.	Press `q` to quit the viewer.
Expecting git diff to show staged changes.	Use `git diff --staged` for those.
Copying a full hash by hand.	The first 7 characters are enough.

Practical exercise

> Edit `README.md` and run `git diff`. Stage it and run `git diff --staged`. Then run `git log --oneline`.

Expected result

The first diff shows your added lines with `+`. After staging, only `--staged` shows them. The log lists your commits.

Knowledge check

1. How do you leave the git log viewer?

- ○ Type exit()
- ○ Press q
- ○ Close Windows

2. Which command shows unstaged changes?

- ○ git diff
- ○ git log
- ○ git show

3. What does git log –oneline show?

- ○ Your passwords
- ○ One short line per commit
- ○ Only the last file

Answers

- Press q — q quits the pager.
- git diff — git diff compares your working files with the staging area.
- One short line per commit — A short hash and the message for each commit.

Summary

git log lists history, git diff and git diff –staged compare changes, and git show inspects one commit.

Key vocabulary

HEAD	The commit you are currently on
git diff	Show unstaged changes
git show	Show a commit and its changes

Next lesson

Module 5 — Connecting Git to GitHub. Your history is local. Next you send it online.

MODULE 04

Practical lab 4 · Create and inspect three commits

1. Run `git add index.html style.css`, then `git commit -m "Add homepage"`.
2. Add the line `p { line-height: 1.6; }` to `style.css`. Check with `git diff`, stage it, then `git commit -m "Improve text spacing"`.
3. Add one sentence to `README.md`. Run `git add README.md` and `git commit -m "Update README"`.
4. Run `git log --oneline` and `git show`.

Check	Pass if…
Three commits	`git log --oneline` lists three lines
Messages	Each starts with a verb and describes the change
Selective staging	Each commit contains only the intended files
Clean state	`git status` says “nothing to commit, working tree clean”

Evaluation: 4 of 4 checks = pass.

Module 4 quiz (pass mark: 70%)

1. Which command stages one file?

- ○ git push index.html
- ○ git log index.html
- ○ git add index.html

2. What does git add . do?

- ○ Uploads to GitHub
- ○ Stages changes in the current folder
- ○ Creates a commit

3. Which command saves staged changes?

- ○ git commit -m "message"
- ○ git add -p
- ○ git diff

4. How do you see staged changes?

- ○ git init
- ○ git diff –staged
- ○ git diff

5. What does (HEAD → main) mean in the log?

- ○ The file is deleted
- ○ The commit is on GitHub
- ○ You are on the latest commit of main

Answers

- git add index.html — git add followed by the file name stages it.
- Stages changes in the current folder — It stages; it does not commit or push.
- git commit -m "message" — commit creates the snapshot.
- git diff –staged — –staged compares the staging area with the last commit.
- You are on the latest commit of main — HEAD marks your current position.